

Institute of science and technology Tribhuvan University



Lab Report of Software Engineering(CSC 375)

SUBMITTED BY:

Shristi Dhakal

Roll no:11 / 79011920

Faculty:BSc.CSIT

Semester:Sixth Semester

SUBMITTED TO:

Nilam Thapa

Department of Computer Science
& Information Techonology,

Prithivi Narayan Campus



Tribhuvan University
त्रिभुवन विश्वविद्यालय

Prithvi Narayan Campus
पृथ्वीनारायण क्याम्पस

Name: Shristi Dhakal

Subject: Software Engineering

Faculty: B.Sc.CSIT/Sixth Semester

Roll no:11/ 79011920

SN	Name of the Experiment	Date of Submission	Remarks
1	Study of Software Types and Attributes of Good Software		
2	Case Study on Software Process Models		
3	Agile Development Simulation		
4	Requirements Analysis		
5	UML System Modeling		
6	Architectural Design		
7	Object-Oriented Design using UML		
8	Testing Techniques		
9	Evolution and Maintenance		
10	Project Management Exercise		

Lab-1

Study of Software Types and Attributes of Good Software

1. Objective: To identify software types and evaluate attributes (usability, reliability, maintainability, efficiency).

2. Theory:

- **Software Types:** Software can be broadly categorized.
 - **System Software:** Manages computer hardware and software resources (e.g., Operating Systems, Device Drivers).
 - **Application Software:** Performs specific tasks for end-users (e.g., Word Processors, Web Browsers, Games).
 - **Programming Software:** Tools for software developers (e.g., Compilers, Debuggers).
- **Attributes of Good Software:** These are non-functional characteristics that describe how well the software performs.
 - **Usability:** How easy and intuitive the software is to learn and use.
 - **Reliability:** The consistency and dependability of the software's performance, including its ability to operate without failure under specified conditions.
 - **Maintainability:** How easy it is to modify, update, and fix bugs in the software.
 - **Efficiency:** How effectively the software utilizes system resources (time, memory, CPU) to perform its functions.

3. Procedure/Steps:

1. Select two real-world software systems.
2. For each selected software system, identify its software type.
3. For each selected software system, evaluate and describe its usability, reliability, maintainability, and efficiency based on common user experience and industry knowledge.
4. Compare the attributes of the two software systems, highlighting similarities and differences.

4. Practical work: Analyze 2 real-world software systems (e.g., MS Word, Facebook) and compare attributes.

Analysis of Microsoft Word

- **Software Type:** Application Software, specifically a word processor. It is designed for creating, editing, formatting, and saving text documents. It falls under the category of general-purpose application software.
- **Attributes:**
 - **Usability:** Microsoft Word is highly usable due to its widely recognized and intuitive graphical user interface (GUI). Features are organized logically in ribbon tabs, and common actions have familiar icons. It offers extensive help documentation, templates, and accessibility features, making it accessible to users with varying levels of computer literacy.
 - **Reliability:** Generally very reliable, especially in stable operating environments. It incorporates robust features like automatic saving and document recovery, which significantly reduce data loss in case of unexpected shutdowns or crashes. While occasional glitches can occur, critical failures are rare.
 - **Maintainability:** Microsoft provides continuous updates, patches, and version upgrades for Word. This includes security fixes, performance enhancements, and new feature additions. Its well-established architecture and development processes contribute to its high maintainability, allowing for long-term support and evolution.
 - **Efficiency:** For its core tasks of text processing and document management, Word is highly efficient. It can handle documents ranging from simple letters to complex reports with embedded media. Its resource consumption is generally optimized for desktop and laptop environments, providing a smooth experience for most users.

Analysis of Facebook

- **Software Type:** Application Software, specifically a social media platform. It is a distributed, web-based, and mobile application designed for social networking, content sharing, and communication among users. It is a specific-purpose application software.
- **Attributes:**
 - **Usability:** Facebook boasts extremely high usability, designed for a massive global user base. Its interface is largely standardized across web and mobile platforms, making it easy to navigate through news feeds, profiles, messaging, and sharing functionalities. The design prioritizes visual content and simple interactions.
 - **Reliability:** Facebook is engineered for high availability and reliability, with vast server infrastructures to ensure continuous service for billions of users. While major outages are rare, local or temporary glitches can occur due to network issues, server load, or ongoing updates. The system is designed to recover quickly from failures.
 - **Maintainability:** Facebook is under constant development and maintenance, with frequent updates rolled out to introduce new features,

improve existing ones, enhance security, and fix bugs. Its microservices architecture and continuous deployment practices support high maintainability, allowing different teams to work on separate components concurrently.

- **Efficiency:** Facebook's backend infrastructure is designed for extreme efficiency in handling massive amounts of real-time data (posts, photos, videos, messages) and user interactions. It efficiently delivers personalized content feeds and facilitates instant communication. User-side efficiency can vary depending on network speed, device specifications, and browser performance.

Comparison:

Attribute	Microsoft Word	Facebook
Software Type	Application Software (Word Processor)	Application Software (Social Media Platform)
Usability	High (Intuitive GUI, feature-rich, extensive help)	High (Simple, accessible, consistent across platforms, designed for mass appeal)
Reliability	High (Stable, robust autosave, infrequent crashes)	High (High uptime for global scale, designed for resilience, occasional glitches)
Maintainability	High (Regular updates, patches, long-term support)	High (Continuous deployment, frequent updates, scalable architecture)
Efficiency	High (Optimized for desktop productivity, resource-efficient)	High (Optimized for large-scale data handling and real-time interactions, network-dependent)

5. Conclusion/Observation:

Both Microsoft Word and Facebook are excellent examples of well-designed software systems, each optimized for its specific purpose and target audience. MS Word showcases strong attributes for individual productivity and document management in a desktop environment, emphasizing stability and user control over content. Facebook, on the other hand, demonstrates high efficiency and reliability in a highly distributed, real-time, and constantly evolving social networking environment, prioritizing scalability and continuous feature delivery. The evaluation of these attributes is crucial in understanding the quality and suitability of software for its intended use.

Lab-2

Case Study on Software Process Models

Objective: To understand different software process models.

Theory/Concept:

Software process models are simplified representations of the software development process. They describe the sequence of activities involved in building software.

- **Waterfall Model:** A linear, sequential approach where each phase (requirements, design, implementation, testing, deployment, maintenance) must be completed before the next begins. It's simple to understand and manage but rigid.
- **Incremental Model:** Divides the system into multiple standalone modules or increments. Each increment is developed and delivered in a series of iterations, with each iteration adding new functionality. This allows for early delivery of basic functionality and feedback incorporation.
- **Prototyping Approach:** Involves creating a preliminary version (prototype) of the software to demonstrate features, gather user feedback, and refine requirements. It's iterative and helps in clarifying uncertain requirements.

Procedure/Steps:

1. Choose a small problem: School Management System.
2. Model the development of a "School Management System" using the Waterfall Model, outlining the phases and activities involved.
3. Model the development of the same system using the Incremental Model, defining several increments and the features included in each.
4. Model the development using the Prototyping Approach, describing the iterative process of prototype creation, feedback, and refinement.
5. Prepare diagrams for each model to visually represent the process.
6. Compare the three models based on their advantages, disadvantages, and suitability for the School Management System.

Practical Work: Model a School Management System with Waterfall Model, Incremental Model, and Prototyping Approach.

Modeling a School Management System

Problem Statement: Develop a School Management System to manage student information, attendance, grades, and basic administrative tasks for a school.

1. Waterfall Model for School Management System

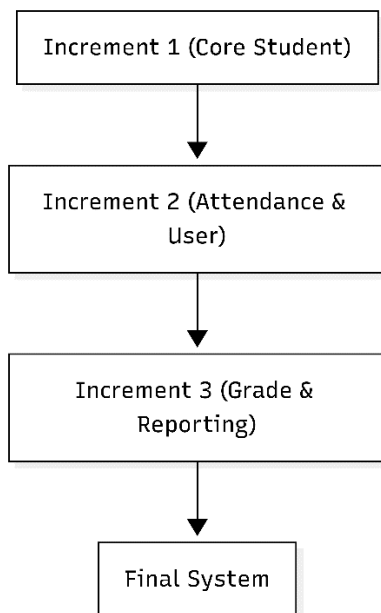
- **Description:** The School Management System would be developed in a strict sequential manner. Each phase must be completed before moving to the next.
- **Phases:**
 1. **Requirements Gathering and Analysis:**
 - Detailed discussions with school administration, teachers, and staff to gather all requirements (e.g., student registration, attendance tracking, grade entry, report generation, basic user roles).
 - Documenting all functional and non-functional requirements.
 - *Output:* Software Requirements Specification (SRS) for the entire system.
 2. **System Design:**
 - Architectural design (e.g., client-server, database schema).
 - Module design (e.g., Student Module, Attendance Module, Grade Module).
 - User interface design.
 - *Output:* Design Document, database schema.
 3. **Implementation (Coding):**
 - Developers write code for each module as per the design.
 - Database creation.
 - Integration of different modules.
 4. **Testing:**
 - Unit testing of individual modules.
 - Integration testing to ensure modules work together.
 - System testing to verify the entire system against requirements.
 - User Acceptance Testing (UAT) by school staff.
 - *Output:* Test reports, bug reports.
 5. **Deployment:**
 - Installation of the system on school servers/computers.
 - User training.
 - Data migration (if any existing student data).
 6. **Maintenance:**
 - Bug fixes, security updates, and minor enhancements after deployment.
- **Diagram:**



2. Incremental Model for School Management System

- **Description:** The School Management System would be built and delivered in increments, with each increment adding more features.

- **Increments:**
 - **Increment 1 (Core Student Management):**
 - **Features:** Student registration, basic student profile (name, ID, contact), search student.
 - **Process:** Requirements -> Design -> Code -> Test -> Deploy (for basic usage/feedback)
 - **Increment 2 (Attendance & User Management):**
 - **Features:** Daily attendance tracking, teacher login, admin login, basic user roles.
 - **Process:** Requirements (for new features) -> Design -> Code -> Test -> Integrate with Inc. 1 -> Deploy
 - **Increment 3 (Grade & Reporting):**
 - **Features:** Grade entry by teachers, student grade viewing, basic report generation (e.g., attendance reports, grade reports).
 - **Process:** Requirements (for new features) -> Design -> Code -> Test -> Integrate with Inc. 1 & 2 -> Deploy
 - **Increment 4 (Advanced Features/Refinements):**
 - **Features:** Fee management, exam scheduling, parent portal, advanced reporting, notification system.
 - **Process:** Requirements -> Design -> Code -> Test -> Integrate -> Deploy (final system)
- **Diagram:**



3. Prototyping Approach for School Management System

- **Description:** An initial, stripped-down version of the system would be built quickly to demonstrate core functionalities and gather user feedback, which then iteratively refines the system.
- **Process:**

1. Basic Requirements Gathering:

- Identify core needs (e.g., student info, attendance). Not all details are required upfront.

2. Quick Design:

- Focus on user interface and main workflows.

3. Prototype Development (Mock-up/Working Model):

- Develop a working prototype for student registration and attendance marking. This might have limited functionality or a simulated backend.

4. User Evaluation/Feedback:

- School administration and teachers use the prototype.
- Feedback is gathered on usability, missing features, and desired changes.
- *Example Feedback:* "We need to filter students by class," "Attendance should also track reasons for absence."

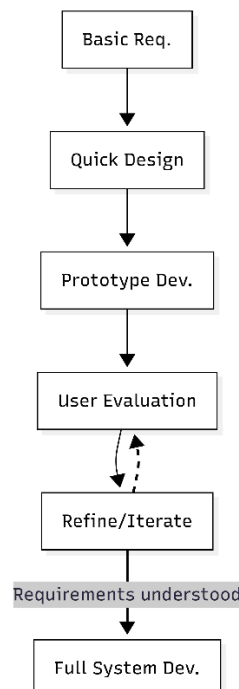
5. Refinement and Iteration:

- The prototype is refined based on feedback.
- New features are added, existing ones modified.
- This loop (Design -> Develop -> Evaluate -> Refine) continues until requirements are sufficiently stable and understood.

6. Full System Development:

- Once the prototype has helped clarify all requirements and design decisions, the full, robust School Management System is developed using a more formal model (e.g., Waterfall or Incremental).

• Diagram:



Output: Comparison of Models for School Management System

Feature/Model	Waterfall Model	Incremental Model	Prototyping Approach
Requirements	Must be fully known upfront.	Evolve over increments; core known upfront.	Evolve and clarify through user feedback.
Risk Handling	High risk if requirements change or are misunderstood later.	Risks are spread across increments, easier to manage.	Early detection of misinterpretations; user satisfaction improves.
Early Delivery	No; system delivered at the very end.	Yes; basic functionality delivered early.	No; prototype delivered for feedback, not final system.
User Involvement	Limited (mainly at requirements and UAT).	Moderate (feedback after each increment).	High and continuous (through prototype evaluation).
Flexibility	Low; difficult to accommodate changes.	Moderate; can adapt to changes in later increments.	High; designed to accommodate changes based on feedback.
Cost	Can be higher if errors found late.	More manageable, costs spread.	Initial prototype cost, but can reduce rework costs.

Conclusion:

We learned about Waterfall, Incremental, and Prototyping software models.

- **Waterfall:** Simple but rigid. Good when requirements are clear from the start.
- **Incremental:** Delivers parts early, easier to handle changes. Good for projects needing continuous feedback.
- **Prototyping:** Helps clarify unclear requirements by showing users early versions. Great for getting user input.

Overall, picking the right model depends on how clear the project requirements are and how much user involvement is needed. For systems like a School Management System, where needs might change, Incremental or Prototyping models are often better than a strict Waterfall approach.

Lab-3

Agile Development Simulation

Objective: To understand Agile principles.

Theory/Concept:

Agile development is an iterative and incremental approach to software development, executed in short cycles called "sprints" or "iterations." Key principles include:

- **Individuals and interactions over processes and tools:** Prioritizing effective communication and collaboration within the team.
- **Working software over comprehensive documentation:** Delivering functional software frequently rather than extensive upfront documentation.
- **Customer collaboration over contract negotiation:** Involving the customer throughout the development process for continuous feedback.
- **Responding to change over following a plan:** Adapting to changes in requirements and technology throughout the project lifecycle.

Core Agile artifacts and events include:

- **Product Backlog:** A prioritized list of all desired features, functionalities, enhancements, and bug fixes for a product. It is dynamic and evolves over time.
- **Sprint Backlog:** A subset of the Product Backlog items selected by the development team to be completed during a specific sprint.
- **Scrum Board (Task Board):** A visual tool used to track the progress of work within a sprint, typically with columns like "To Do," "In Progress," and "Done."

Procedure/Steps:

1. Identify the main features for an "Online Food Delivery System" to create a Product Backlog.
2. Prioritize these features.
3. Define the scope for two "Sprints," creating a Sprint Backlog for each.
4. Simulate the progress of tasks within these sprints using a Scrum Board.

Practical Work:

Problem: Online Food Delivery System

1. Product Backlog for an Online Food Delivery System:

ID	Feature (User Story)	Priority	Estimation (Story Points)	Notes
PB1	As a user, I want to register and create a profile.	High	5	Basic user account creation
PB2	As a user, I want to search for restaurants.	High	8	By name, cuisine, location
PB3	As a user, I want to view restaurant menus.	High	5	With item details and prices
PB4	As a user, I want to add items to my cart.	High	3	Quantity selection
PB5	As a user, I want to proceed to checkout.	High	8	Review order, apply discounts
PB6	As a user, I want to pay online (credit/debit).	High	13	Secure payment gateway integration
PB7	As a user, I want to track my order status.	Medium	8	Real-time updates
PB8	As a user, I want to view my order history.	Medium	5	Previous orders details
PB9	As a user, I want to apply promotional codes.	Medium	3	Discount application
PB10	As a restaurant, I want to manage my menu.	Medium	13	Add, edit, remove items
PB11	As a restaurant, I want to view incoming orders.	High	8	Order details and acceptance
PB12	As a delivery driver, I want to receive delivery requests.	Medium	8	View order details, pick-up/drop-off
PB13	As a restaurant, I want to update order status.	Medium	5	E.g., preparing, ready for pickup

ID	Feature (User Story)	Priority	Estimation (Story Points)	Notes
PB14	As an admin, I want to manage users and restaurants.	Low	13	Basic CRUD operations
PB15	As a user, I want to leave reviews for restaurants.	Low	5	Rating and text comments

2. Sprint Backlogs:

Sprint 1 (Focus: Core User Experience)

- **Duration:** 2 Weeks
- **Sprint Goal:** Enable users to find restaurants, view menus, add items to cart, and prepare for checkout.

ID	Feature (User Story)	Priority (from PB)	Est. (Story Points)
SP1.1	As a user, I want to register and create a profile.	High	5
SP1.2	As a user, I want to search for restaurants.	High	8
SP1.3	As a user, I want to view restaurant menus.	High	5
SP1.4	As a user, I want to add items to my cart.	High	3

Sprint 2 (Focus: Payment & Basic Restaurant Management)

- **Duration:** 2 Weeks
- **Sprint Goal:** Enable users to complete orders with online payment, and allow restaurants to receive orders.

ID	Feature (User Story)	Priority (from PB)	Est. (Story Points)
SP2.1	As a user, I want to proceed to checkout.	High	8

ID	Feature (User Story)	Priority (from PB)	Est. (Story Points)
SP2.2	As a user, I want to pay online (credit/debit).	High	13
SP2.3	As a restaurant, I want to view incoming orders.	High	8

3. Scrum Board Simulation:

Sprint 1 Scrum Board

To Do	In Progress	Done
		SP1.1: Register/Create Profile
	SP1.2: Search Restaurants	
SP1.3: View Restaurant Menus		
SP1.4: Add Items to Cart		

Simulation Progress (Mid-Sprint 1):

SP1.1 is completed. SP1.2 is currently being worked on. SP1.3 and SP1.4 are pending.

Sprint 2 Scrum Board

To Do	In Progress	Done
		SP2.1: Proceed to Checkout
SP2.2: Pay Online	SP2.3: View Incoming Orders	

Simulation Progress (Mid-Sprint 2):

SP2.1 is completed. SP2.3 is currently being worked on. SP2.2 is pending.

Conclusion/Observation:

This lab provided a hands-on simulation of Agile development principles using a hypothetical Online Food Delivery System. By creating a Product Backlog, we learned how to gather and prioritize features for a complex system. The subsequent Sprint

Backlogs demonstrated how work is broken down into manageable, time-boxed iterations, focusing on delivering specific value in each sprint. The Scrum Board visually illustrated the flow of work, from "To Do" to "Done," emphasizing transparency and facilitating daily stand-ups to track progress and identify impediments. This exercise highlighted the iterative nature of Agile, its flexibility in adapting to evolving requirements, and its focus on continuous delivery of working software and customer collaboration. Understanding these concepts is crucial for managing projects in dynamic environments.

Lab-4

Requirements Analysis

Objective: To practice functional and non-functional requirements.

Theory/Concept:

Requirements analysis is a crucial phase in software engineering where the needs of stakeholders are identified and documented.

- **Functional Requirements (FRs):** Describe what the system *does*. These are actions that the system must perform. They define the behavior of the software.
- **Non-Functional Requirements (NFRs):** Describe *how* the system performs. These are quality attributes that constrain the design and implementation, such as performance, security, usability, reliability, scalability, and maintainability.
- **Use Case Diagram:** A Unified Modeling Language (UML) diagram that visually represents the functional requirements of a system. It shows actors (users or external systems) and use cases (functions the system performs) and their relationships.

Procedure/Steps:

1. Understand the problem statement: "Student Attendance System."
2. Identify all possible functional requirements for the system from the perspective of different users (e.g., student, teacher, admin).
3. Identify relevant non-functional requirements that would ensure the quality and performance of the system.
4. Draw a Use Case Diagram to visually represent the main functional requirements and actors interacting with the system.

Practical Work: Write SRS for a "Student Attendance System" including Functional requirements, Non-functional requirements, and Use Case Diagram.

Software Requirements Specification (SRS) for a Student Attendance System

1. Introduction

This document outlines the functional and non-functional requirements for a Student Attendance System. The system aims to automate the process of recording, managing, and reporting student attendance for educational institutions.

2. Functional Requirements (FRs)

The system shall provide the following functionalities:

2.1. User Management:

- FR1.1: The system shall allow an Administrator to add new teachers and students.
- FR1.2: The system shall allow an Administrator to edit existing teacher and student profiles.
- FR1.3: The system shall allow an Administrator to deactivate/delete teacher and student accounts.
- FR1.4: The system shall allow all users (Teacher, Student, Admin) to log in securely using a username and password.
- FR1.5: The system shall allow users to retrieve forgotten passwords.

2.2. Class/Course Management:

- FR2.1: The system shall allow an Administrator to add and manage courses/subjects.
- FR2.2: The system shall allow an Administrator to assign courses to specific teachers.
- FR2.3: The system shall allow an Administrator to enroll students in specific courses.

2.3. Attendance Marking:

- FR3.1: The system shall allow a Teacher to mark attendance for students in their assigned courses.
- FR3.2: The system shall support marking attendance as "Present," "Absent," "Late," or "Excused."
- FR3.3: The system shall allow a Teacher to edit attendance records for a specific date and class.
- FR3.4: The system shall automatically timestamp attendance entries.

2.4. Attendance Viewing and Reporting:

- FR4.1: The system shall allow a Student to view their attendance record for all enrolled courses.
- FR4.2: The system shall allow a Teacher to view attendance summaries for their assigned courses.
- FR4.3: The system shall allow an Administrator to view attendance records for any student or class.
- FR4.4: The system shall generate daily, weekly, and monthly attendance reports (e.g., percentage attendance, absent lists).
- FR4.5: The system shall allow reports to be exported in common formats (e.g., PDF, Excel).

3. Non-Functional Requirements (NFRs)

3.1. Performance:

- NFR3.1.1: The system shall load any page within 3 seconds under normal network conditions.
- NFR3.1.2: Attendance marking for a class of 50 students shall be processed within 2 seconds.
- NFR3.1.3: Reports for a semester's attendance for 500 students shall be generated within 10 seconds.

3.2. Security:

- NFR3.2.1: The system shall use encrypted communication (HTTPS) for all data transmission.
- NFR3.2.2: User passwords shall be securely hashed and salted.
- NFR3.2.3: Role-based access control (RBAC) shall be enforced (e.g., students cannot modify grades).
- NFR3.2.4: The system shall be protected against common web vulnerabilities (e.g., SQL injection, XSS).

3.3. Usability:

- NFR3.3.1: The user interface shall be intuitive and easy to navigate for all user roles.
- NFR3.3.2: Error messages shall be clear, concise, and helpful.
- NFR3.3.3: The system shall provide a consistent look and feel across all its modules.

3.4. Reliability:

- NFR3.4.1: The system shall ensure data integrity for all attendance records.
- NFR3.4.2: The system shall maintain 99.5% uptime during school operating hours.
- NFR3.4.3: All data shall be backed up daily to prevent data loss.

3.5. Scalability:

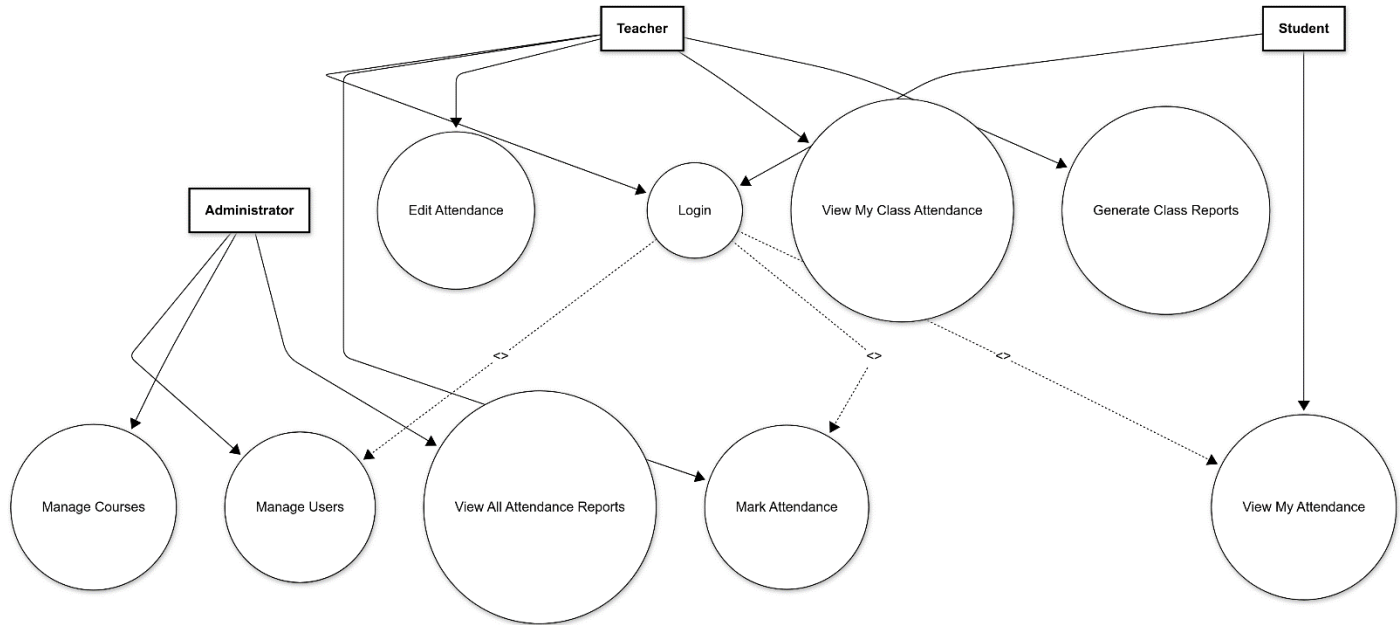
- NFR3.5.1: The system shall be able to handle up to 5000 students and 200 teachers without significant performance degradation.
- NFR3.5.2: The system architecture shall support future expansion of features (e.g., parental portal, notification system).

3.6. Maintainability:

- NFR3.6.1: The system code shall be modular and well-documented.

- NFR3.6.2: New features and bug fixes should be implementable with minimal effort.

4. Use Case Diagram for Student Attendance System



Conclusion/Observation:

This lab provided a practical exercise in defining software requirements, distinguishing between functional and non-functional aspects. By drafting an SRS for a "Student Attendance System," we identified specific functionalities required by different users (Administrator, Teacher, Student) and outlined critical quality attributes like performance, security, and usability. The creation of a Use Case Diagram visually demonstrated the interactions between actors and the system's core functions, offering a clear, high-level overview of the system's behavior. This exercise highlighted the importance of a comprehensive requirements analysis phase in software development to ensure that the final product meets user needs and quality expectations.

Lab-5

UML System Modeling

Objective: To create UML diagrams.

Theory/Concept:

UML (Unified Modeling Language) is a standardized visual language used to model software systems. It provides various diagrams to represent different aspects of a system.

- **Context Diagram:** A high-level data flow diagram that shows the system as a single process, its external entities (actors/terminators), and the major data flows between the system and these entities. It defines the system's boundary.
- **Use Case Diagram:** (Reviewed in Lab 4) Shows the functional requirements of a system in terms of actors and use cases.
- **Sequence Diagram:** A UML interaction diagram that shows the order of interactions between objects in a sequential manner. It depicts the objects involved in the scenario and the sequence of messages exchanged between them to perform a specific function. Lifelines represent objects, and arrows represent messages.
- **Activity Diagram:** A UML behavioral diagram that depicts the dynamic aspects of the system. It shows the flow of control or data from one activity to another, representing workflows, business processes, or operation steps. It uses nodes like action states, control flows, decision nodes, and fork/join nodes.

Procedure/Steps:

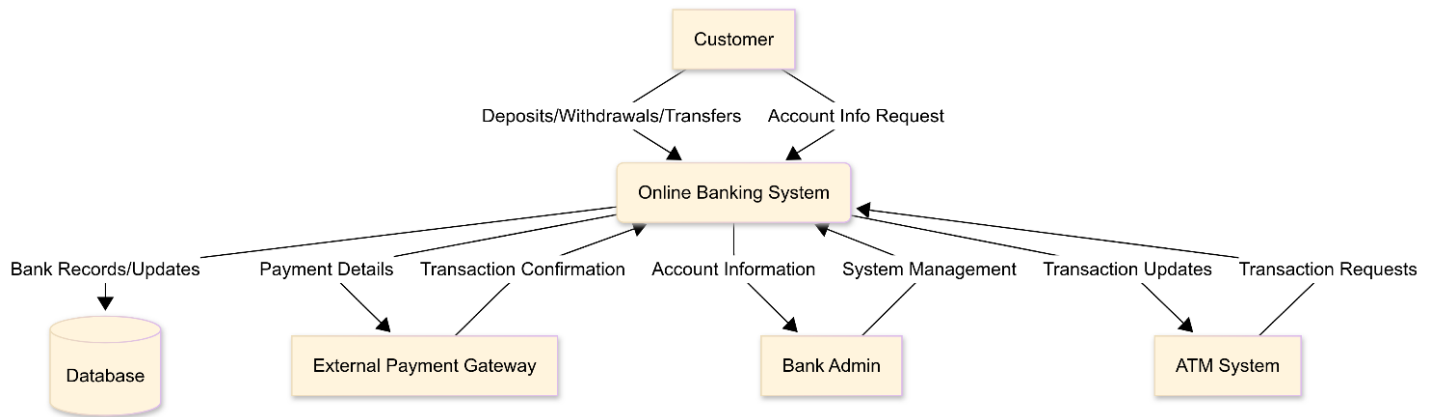
1. Understand the problem statement: "Online Banking System."
2. For the Online Banking System, draw a Context Diagram, identifying the system, external entities, and major data flows.
3. For the Online Banking System, draw a Use Case Diagram, identifying actors and their interactions with the system's core functions.
4. Select a specific scenario (e.g., "Transfer Funds") and draw a Sequence Diagram to illustrate the message flow between objects.
5. Select a specific process (e.g., "Process Loan Application") and draw an Activity Diagram to show the workflow.

Practical Work: For an **Online Banking System**, prepare Context Diagram, Use Case Diagram, Sequence Diagram, and Activity Diagram.

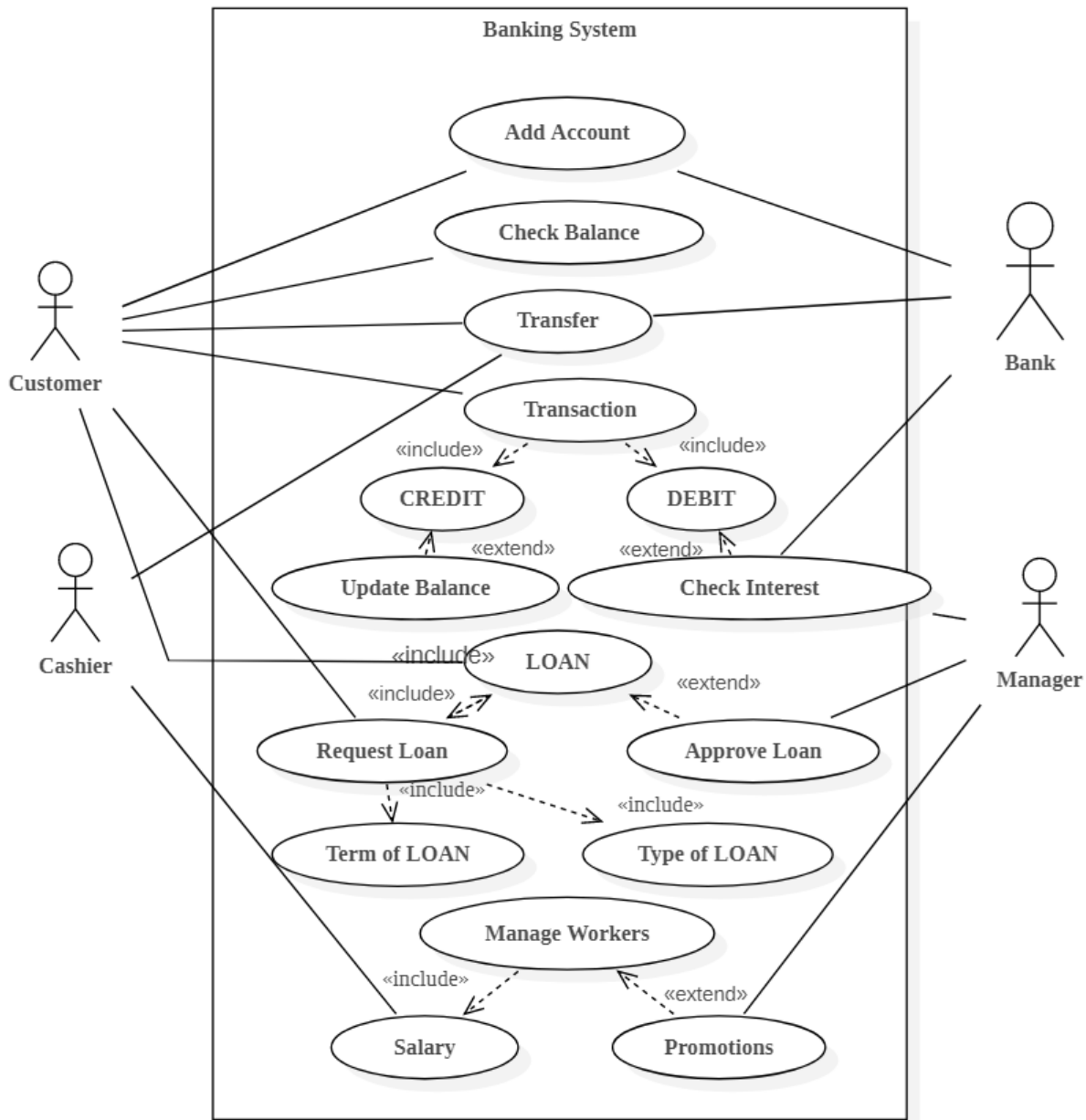
Modeling an Online Banking System

Problem: Online Banking System

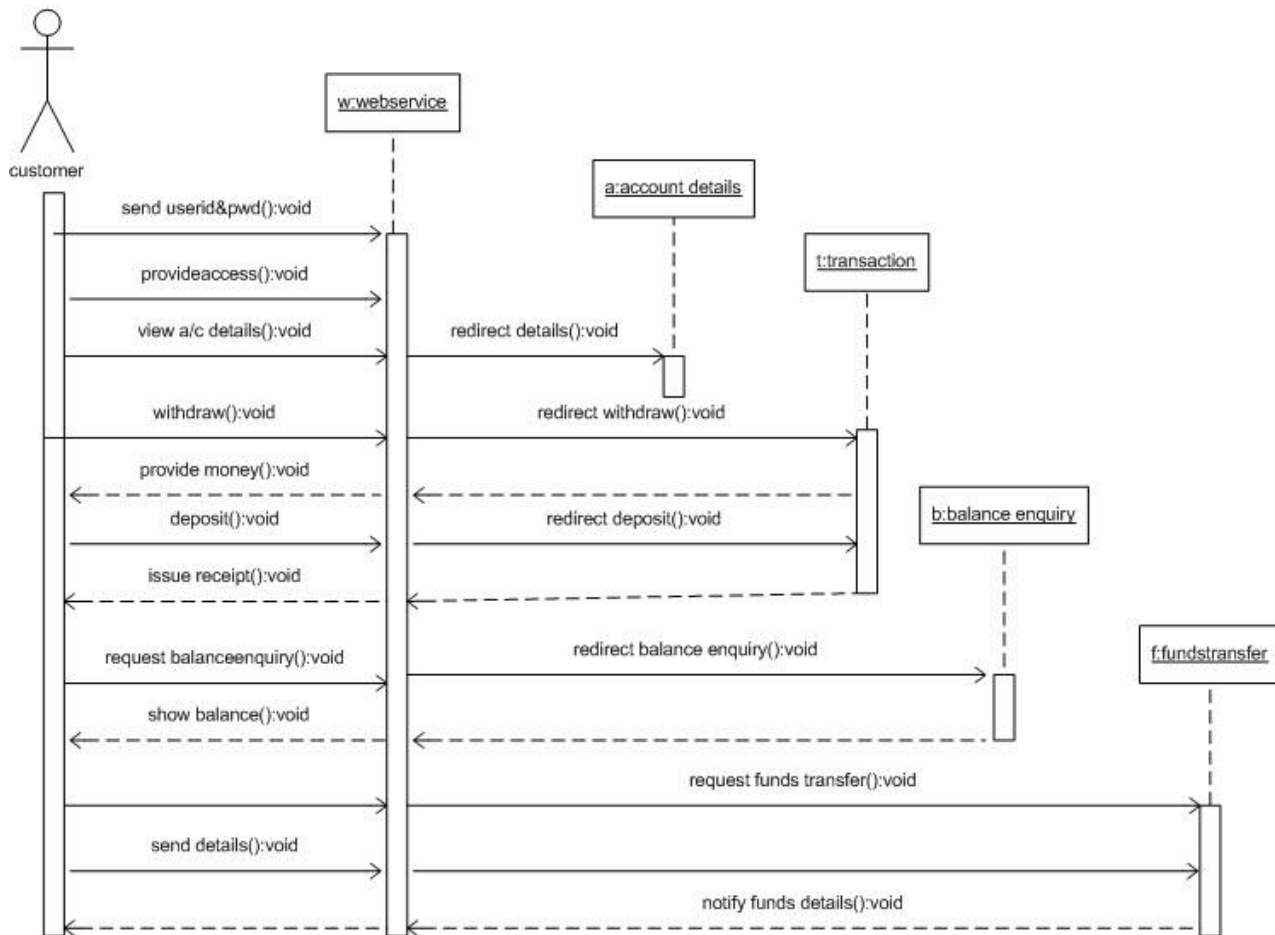
1. Context Diagram for Online Banking System



2. Use Case Diagram for Online Banking System

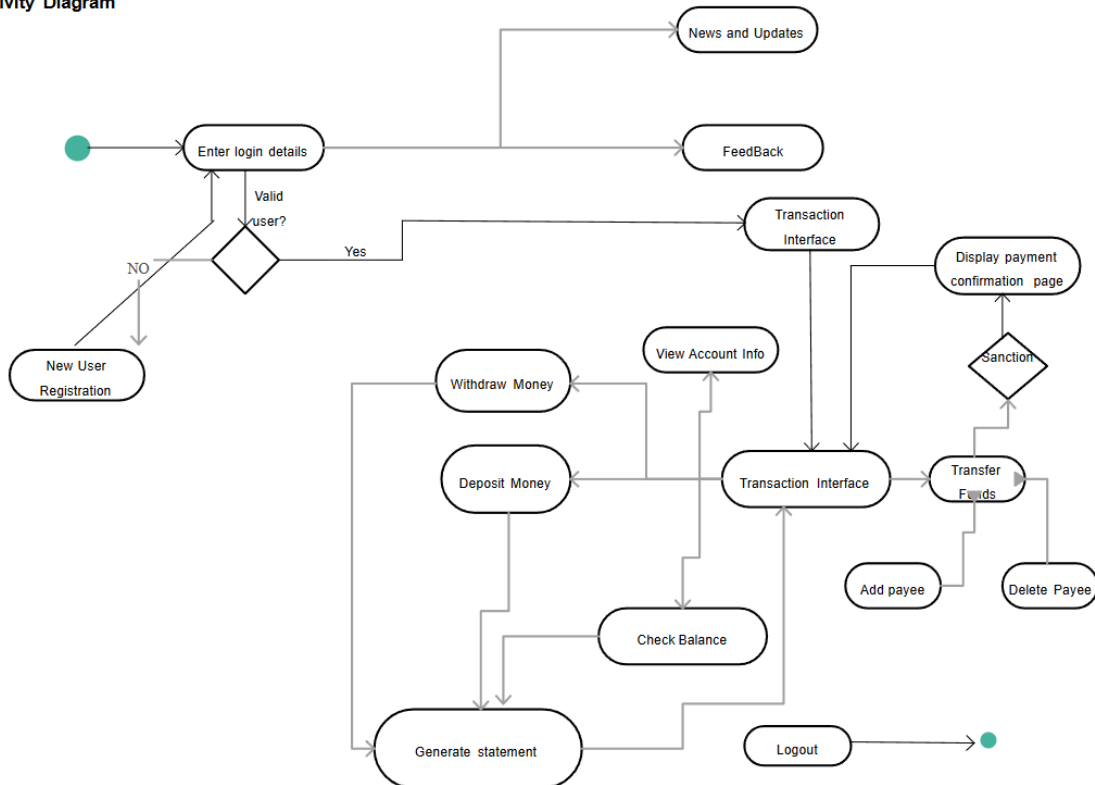


3. Sequence Diagram for Online Banking System



4. Activity Diagram for "Process Loan Application"

Activity Diagram



Conclusion/Observation:

This lab provided hands-on experience in creating various UML diagrams for an Online Banking System. The **Context Diagram** helped define the system's boundaries and its primary interactions with external entities. The **Use Case Diagram** visually represented the system's functional requirements from the perspective of different actors, providing a high-level overview of what the system does. By developing a **Sequence Diagram** for "Transfer Funds," we demonstrated the detailed interaction flow between different objects within the system, crucial for understanding communication patterns. Finally, the **Activity Diagram** for "Process Loan Application" illustrated the workflow and decision points involved in a complex business process. This exercise reinforced the importance of UML as a powerful tool for visualizing, specifying, constructing, and documenting the artifacts of a software system, aiding in clear communication and design before implementation.

Lab-6

Architectural Design

Objective: To explore software architecture.

Theory/Concept:

Software architecture defines the fundamental structure of a software system and provides a conceptual model for its components, their relationships, and the principles governing their design and evolution.

- **Layered Architecture:** Organizes the system into horizontal layers, where each layer provides services to the layer above it and uses services from the layer below. Common layers include Presentation (UI), Application (business logic), Domain (core business objects), and Data Access (database interaction). This promotes separation of concerns.
- **Client-Server Architecture:** A distributed architecture where the system is divided into two main parts: clients (requesting services) and servers (providing services). Clients communicate with servers to access shared resources or perform operations.
- **Design Pattern:** A general, reusable solution to a commonly occurring problem within a given context in software design. They are not finished designs that can be transformed directly into code, but rather templates for how to solve a problem that can be used in many different situations.
 - **Model-View-Controller (MVC):** A popular architectural pattern for designing user interfaces, dividing an application into three interconnected components:
 - **Model:** Manages the application's data, logic, and rules.
 - **View:** Presents data to the user.
 - **Controller:** Receives user input, manipulates the Model, and updates the View.

Procedure/Steps:

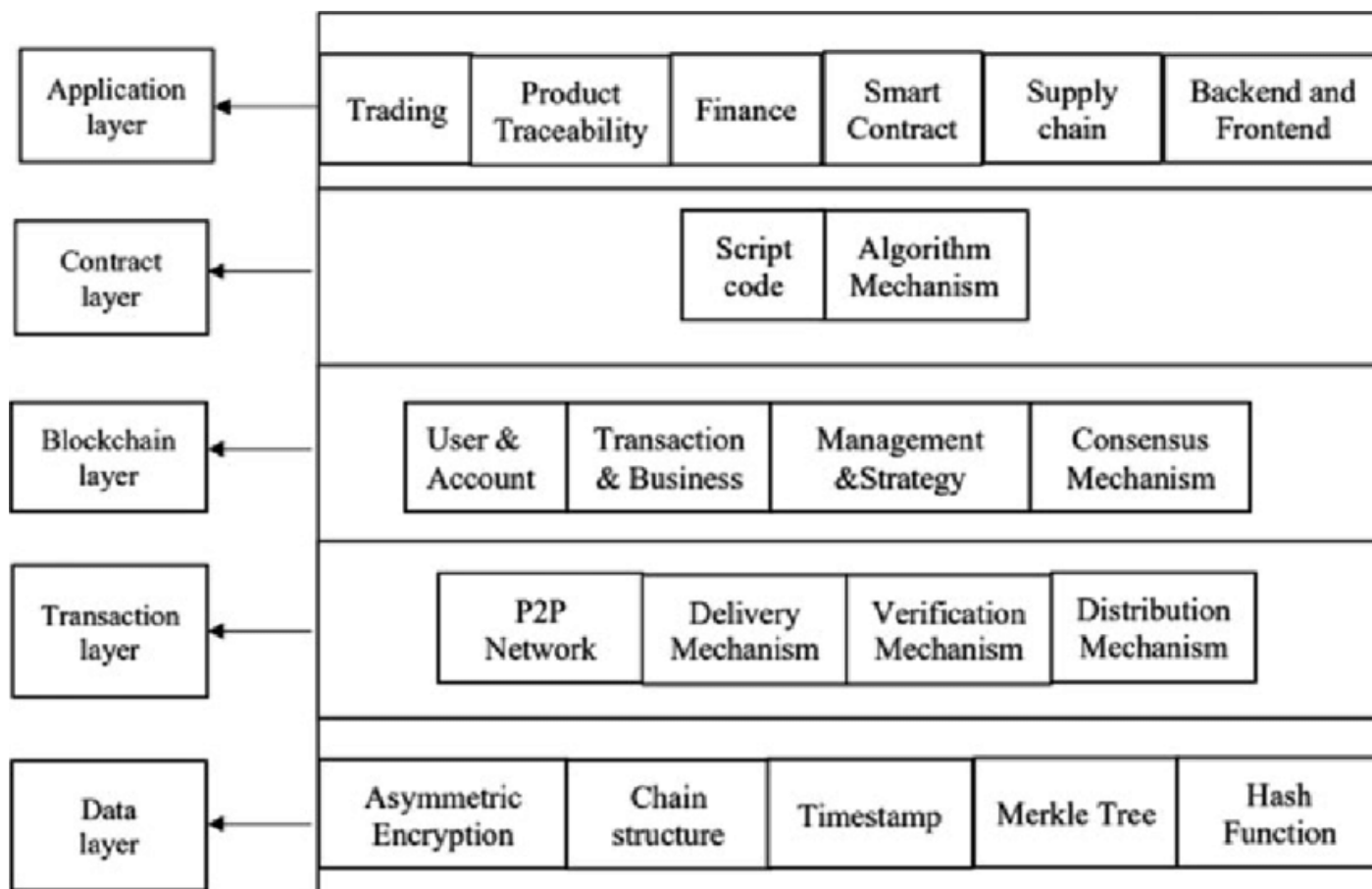
1. Understand the problem statement: "E-commerce Website."
2. For the E-commerce Website, prepare a Layered Architecture Diagram, outlining the typical layers involved.
3. For the E-commerce Website, prepare a Client-Server Architecture Diagram, identifying clients, servers, and communication flow.
4. Identify and briefly describe a suitable design pattern (e.g., MVC) for the E-commerce Website, explaining how it applies.

Practical Work: For an **E-commerce Website**, prepare Layered Architecture Diagram, Client-Server Architecture Diagram, and identify suitable design pattern (e.g., MVC).

Architectural Design for an E-commerce Website

Problem: E-commerce Website

1. Layered Architecture Diagram for E-commerce Website



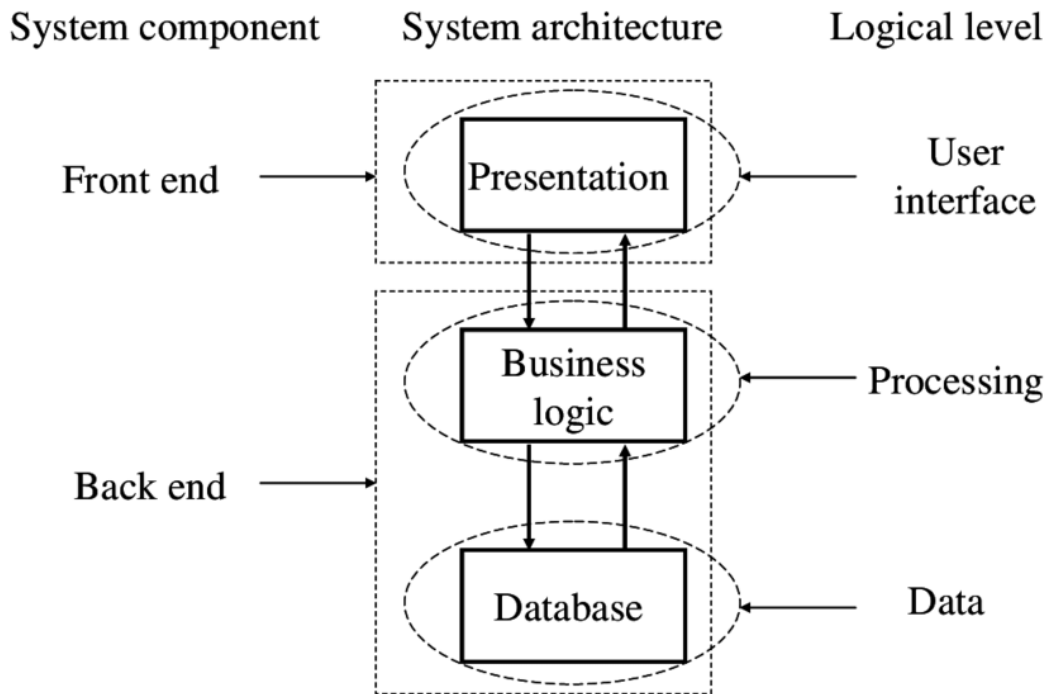
Explanation:

1. **Data Layer** – Provides the foundation with **encryption, hashing, timestamps, chain structure, and Merkle trees** to secure and store data reliably.
2. **Transaction Layer** – Handles **P2P networking, delivery, verification, and distribution of transactions**, ensuring smooth communication between nodes.
3. **Blockchain Layer** – Manages **users, accounts, transactions, business rules, and consensus mechanisms** to maintain trust and agreement.
4. **Contract Layer** – Executes **smart contracts and algorithms**, automating business logic like payments, refunds, or supply chain rules.
5. **Application Layer** – The user-facing part for **trading, finance, product traceability, supply chain management, backend & frontend operations**.

In short:

- Data layer secures,
- Transaction layer transmits,
- Blockchain layer manages,
- Contract layer automates,
- Application layer serves users.

2. Client-Server Architecture Diagram for E-commerce Website



Explanation:

1. **Presentation Layer (Front end / User Interface)**
 - What customers see (website or app).
 - Displays products, shopping cart, order details.
2. **Business Logic Layer (Processing / Server side)**
 - Handles rules and processing.
 - Manages login, payments, order confirmation, stock checking.
3. **Database Layer (Back end / Data)**
 - Stores all data like user accounts, products, orders, and payments.

Flow:

Customer request (frontend) → processed by server logic → fetch/store in database → response back to user.

3. Suitable Design Pattern: Model-View-Controller (MVC)

MVC is an excellent design pattern for an E-commerce Website, especially for the presentation and application layers.

How it applies:

- **Model:** In an e-commerce context, the Model would represent the data and business logic related to entities like Product, User, Order, Cart, Payment. It would contain the logic for calculating prices, managing inventory, validating user input, and interacting with the database (via the Data Access Layer).
- **View:** The View would be the user interface elements seen by the customer (e.g., product listing pages, product detail pages, shopping cart page, checkout form, order history page). These views would display data retrieved from the Model.
- **Controller:** The Controller would handle user interactions. For example, when a user clicks "Add to Cart," the Controller receives this request, tells the Model to update the cart, and

then tells the View to refresh to show the updated cart contents. It acts as an intermediary between the Model and the View.

Advantages for E-commerce:

- **Separation of Concerns:** Clearly separates data/logic from presentation, making the codebase easier to manage and test.
- **Reusability:** Models can be reused with different views (e.g., web view, mobile app view).
- **Maintainability:** Changes to the UI don't necessarily impact the business logic, and vice versa.
- **Testability:** Each component can be tested independently.

Conclusion/Observation:

This lab provided a fundamental understanding of software architecture concepts by applying them to an E-commerce Website. We illustrated a **Layered Architecture**, demonstrating how systems can be logically separated into distinct tiers, promoting modularity, maintainability, and scalability. The **Client-Server Architecture** diagram depicted the distributed nature of the system, highlighting the interaction between user-facing clients and backend servers. Finally, by identifying **MVC** as a suitable design pattern, we understood how to organize the application's components for better separation of concerns, leading to more manageable, testable, and maintainable code, particularly in the context of web applications with rich user interfaces. This exercise underscored the importance of thoughtful architectural design in building robust, scalable, and adaptable software systems.

Lab-7

Object Oriented Design using UML

Objective: To practice OOD principles.

Theory/Concept:

Object-Oriented Design (OOD) is a software design approach that structures a system into interacting objects. Key principles include:

- **Encapsulation:** Bundling data (attributes) and methods (behaviors) that operate on the data within a single unit or class.
- **Inheritance:** A mechanism where one class (subclass/derived class) acquires the properties and behaviors of another class (superclass/base class).
- **Polymorphism:** The ability of an object to take on many forms. In OOD, it means a single interface can be used for different data types.
- **Abstraction:** Hiding complex implementation details and showing only the essential features of an object.

UML diagrams used in OOD:

- **Class Diagram:** A static UML diagram that shows the classes in a system, their attributes (data), operations (methods), and the relationships between them (e.g., association, aggregation, composition, inheritance).
- **Object Diagram:** A static UML diagram that shows a snapshot of the system at a particular point in time, illustrating instances of classes (objects) and their relationships. It represents a concrete example of a class diagram.

Procedure/Steps:

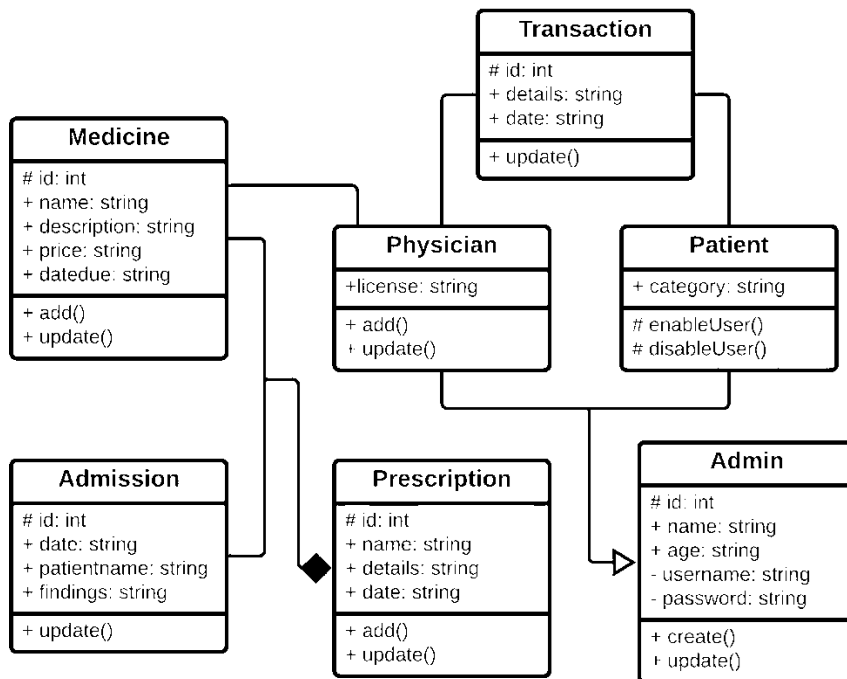
1. Understand the problem statement: "Hospital Management System."
2. Identify the main entities (classes) in a Hospital Management System (e.g., Patient, Doctor, Appointment, Department).
3. For these classes, define their attributes and methods.
4. Identify relationships between these classes (e.g., a Patient has Appointments, a Doctor works in a Department).
5. Draw a Class Diagram to represent these classes and their relationships.
6. Create a simple scenario (e.g., a specific patient having an appointment with a specific doctor) and draw an Object Diagram to illustrate instances of the classes.
7. Select a sample module (e.g., Patient Registration) and implement it in a programming language (Java/Python/C++) to demonstrate basic OOD principles.

Practical Work: For a **Hospital Management System**, create Class Diagram, Object Diagram, and implement a sample module (e.g., Patient Registration) in any programming language.

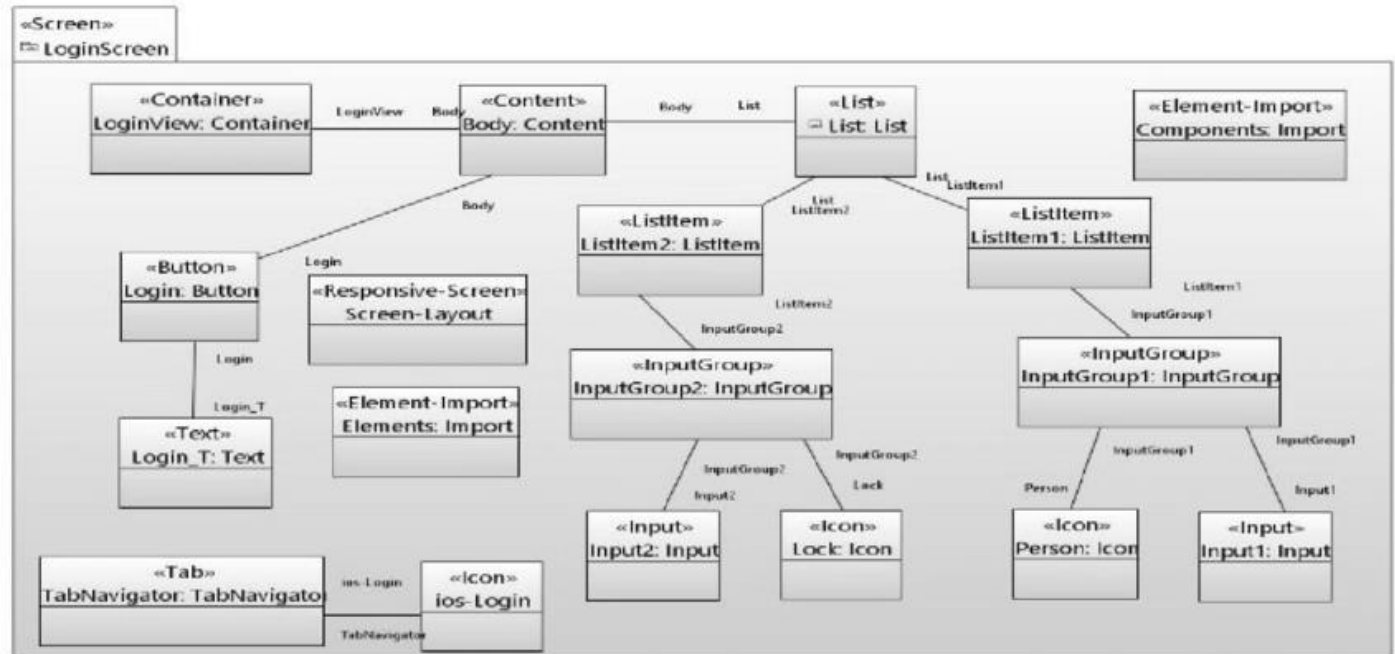
Object-Oriented Design for a Hospital Management System

Problem: Hospital Management System

1. Class Diagram for Hospital Management System



2. Object Diagram for a Specific Scenario (Login Screen)



3. Implementation of a Sample Module (Patient Registration) in Python

```
import datetime
```

```
class Person:
```

```
    def __init__(self, name, address, phone, dob):
        self.name = name
        self.address = address
        self.phone = phone
        self.dob = dob # Date of Birth
```

```
    def get_details(self):
```

```
        return f"Name: {self.name}, Address: {self.address}, Phone: {self.phone}, DOB: {self.dob}"
```

```
class Patient(Person):
```

```
next_patient_id = 1000
```

```
def __init__(self, name, address, phone, dob, medical_history="N/A"):  
    super().__init__(name, address, phone, dob)  
    self.patient_id = self._generate_patient_id()  
    self.medical_history = medical_history  
    self.appointments = [] # List to store Appointment objects
```

```
def _generate_patient_id(self):  
    Patient.next_patient_id += 1  
    return f"P{Patient.next_patient_id:04d}"
```

```
def register(self):  
    print(f"Patient {self.name} registered successfully with ID: {self.patient_id}")
```

```
def view_details(self):  
    print("\n--- Patient Details ---")  
    print(f"Patient ID: {self.patient_id}")  
    print(self.get_details())  
    print(f"Medical History: {self.medical_history}")  
    if self.appointments:  
        print("--- Upcoming Appointments ---")  
        for appt in self.appointments:  
            print(f" - ID: {appt.appointment_id}, Date: {appt.appt_date}, Time: {appt.appt_time},  
Status: {appt.status}")  
    else:  
        print("No upcoming appointments.")
```

```
def add_appointment(self, appointment):  
    self.appointments.append(appointment)
```

```
class Doctor(Person):  
    next_doctor_id = 2000
```

```
def __init__(self, name, address, phone, dob, specialization, license_no):  
    super().__init__(name, address, phone, dob)  
    self.doctor_id = self._generate_doctor_id()  
    self.specialization = specialization  
    self.license_no = license_no  
    self.patients_assigned = []  
    self.appointments_scheduled = []
```

```
def _generate_doctor_id(self):  
    Doctor.next_doctor_id += 1  
    return f"D{Doctor.next_doctor_id:04d}"
```

```
def view_details(self):  
    print("\n--- Doctor Details ---")  
    print(f"Doctor ID: {self.doctor_id}")  
    print(self.get_details())  
    print(f"Specialization: {self.specialization}")
```

```

        print(f"License No: {self.license_no}")

class Appointment:
    next_appointment_id = 3000

    def __init__(self, appt_date, appt_time, patient, doctor, status="Scheduled"):
        self.appointment_id = self._generate_appointment_id()
        self.appt_date = appt_date
        self.appt_time = appt_time
        self.patient = patient
        self.doctor = doctor
        self.status = status

    def _generate_appointment_id(self):
        Appointment.next_appointment_id += 1
        return f"A{Appointment.next_appointment_id:04d}"

# ----- Testing the Code -----

# Create patient and doctor
p1 = Patient("Alice", "Kathmandu", "9812345678", "1998-05-20", "Allergy")
d1 = Doctor("Dr. Bob", "Pokhara", "9801112233", "1975-02-10", "Cardiologist", "LIC1234")

# Register patient
p1.register()

# Create appointment
appt1 = Appointment("2025-09-01", "10:30 AM", p1, d1)

# Link appointment
p1.add_appointment(appt1)
d1.appointments_scheduled.append(appt1)

# View details
p1.view_details()
d1.view_details()

```

Output:

Patient Alice registered successfully with ID: P1001

--- Patient Details ---

Patient ID: P1001

Name: Alice, Address: Kathmandu, Phone: 9812345678, DOB: 1998-05-20

Medical History: Allergy

--- Upcoming Appointments ---

- ID: A3001, Date: 2025-09-01, Time: 10:30 AM, Status: Scheduled

--- Doctor Details ---

Doctor ID: D2001

Name: Dr. Bob, Address: Pokhara, Phone: 9801112233, DOB: 1975-02-10

Specialization: Cardiologist

License No: LIC1234

Conclusion/Observation:

This lab provided a practical exercise in applying Object-Oriented Design (OOD) principles to a Hospital Management System. Through the creation of a **Class Diagram**, we learned to identify the key entities (classes), their attributes and methods, and the relationships between them, which is fundamental to structuring an object-oriented system. The **Object Diagram** allowed us to visualize a concrete instance of these classes and their interactions at a specific point in time, reinforcing the concepts of class instantiation and relationships. Furthermore, implementing a sample module like **Patient Registration** in Python demonstrated how OOD principles such as encapsulation and inheritance are applied in actual code, providing a hands-on understanding of how objects interact and manage data. This lab underscored the value of OOD in creating modular, reusable, and maintainable software systems, which is essential for complex applications like a Hospital Management System.

Lab-8

Testing Techniques

Objective: To apply testing strategies.

Theory:

Software testing is a crucial part of software development that ensures the software works correctly and meets its requirements. It involves various strategies and techniques to identify defects and ensure quality.

- **Unit Testing:** Testing individual components or modules of the software in isolation. The goal is to verify that each unit of the software performs as designed.
- **Black Box Testing:** A testing method where the internal structure, design, and implementation of the item being tested are not known to the tester. Testers evaluate the software's functionality from an external perspective, based solely on inputs and outputs.
- **White Box Testing:** A testing method where the internal structure, design, and implementation of the item being tested are known to the tester. Testers examine the code, internal paths, and logic to ensure all parts of the code are thoroughly exercised.
- **Test Case Table:** A document used to organize and record test cases, which specify inputs, expected outputs, and execution conditions for a particular test.

Procedure/Steps:

1. **Unit Tests for a Simple Calculator Program:**
 - Design a simple calculator program with addition and subtraction functionalities.
 - Write unit tests for the addition and subtraction functions to verify their correctness for various inputs.
2. **Black Box Testing:**
 - For the simple calculator program, identify test cases without looking at the internal code.
 - Consider different input scenarios, including valid, invalid, and edge cases, for both addition and subtraction.
 - Record the inputs, expected outputs, and actual outputs.
3. **White Box Testing:**
 - Examine the source code of the calculator program.
 - Design test cases to ensure all paths in the code (e.g., if-else statements, loops) for addition and subtraction are covered.
 - Record the inputs, expected outputs, and actual outputs.
4. **Prepare a Test Case Table:**
 - Consolidate all designed test cases into a structured table, including columns for Test Case ID, Description, Input, Expected Output, Actual Output, and Pass/Fail Status.

Practical Work (Example/Diagram/Code):

(Example for Unit Tests for a Simple Calculator Program in Python)

```
# calculator.py
```

```
def add(a, b):  
    return a + b
```

```
def subtract(a, b):  
    return a - b
```

```
# test_calculator.py (using Python's unittest module)
import unittest
from calculator import add, subtract
```

```
class TestCalculator(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(2, 3), 5)
        self.assertEqual(add(-1, 1), 0)
        self.assertEqual(add(-1, -1), -2)
        self.assertEqual(add(0, 0), 0)

    def test_subtract(self):
        self.assertEqual(subtract(5, 2), 3)
        self.assertEqual(subtract(2, 5), -3)
        self.assertEqual(subtract(0, 0), 0)
        self.assertEqual(subtract(10, -5), 15)
```

```
if __name__ == '__main__':
    unittest.main()
```

(Example for Black Box Testing)

Test Case ID	Description	Input (a, b)	Operation	Expected Output	Actual Output	Pass/Fail
BB-ADD-001	Positive numbers addition	(5, 3)	Add	8		
BB-SUB-001	Positive numbers subtraction	(10, 4)	Subtract	6		
BB-ADD-002	Zero addition	(0, 7)	Add	7		
BB-SUB-002	Subtracting from zero	(0, 5)	Subtract	-5		

(Example for White Box Testing)

Considering the simple `add` and `subtract` functions above, white box testing would involve ensuring that the `return a + b` and `return a - b` lines are executed. For such simple functions, the unit tests already cover the code paths. For more complex functions with conditional logic or loops, white box testing would focus on creating test cases that specifically hit each branch and loop iteration.

Conclusion/Observation:

In this lab, we successfully applied various software testing techniques. We performed unit tests to verify the correctness of individual functions of a calculator program. We then conducted black box testing by designing test cases based on external specifications, and white box testing by considering the internal code structure to ensure code coverage. The creation of a test case table helped in systematically documenting the test process, which is essential for comprehensive testing and bug tracking. These exercises demonstrated the importance of different testing strategies in ensuring the reliability and quality of software.

Lab-9

Evolution and Maintenance

Objective: To understand software evolution and practice documenting maintenance activities.

Theory:

Software evolution refers to the continuous process of developing, modifying, and improving software over its lifespan. This involves adapting the software to changing requirements, fixing defects, enhancing performance, and adding new functionalities. Software maintenance is a key part of this evolution, categorizing into:

- **Corrective Maintenance:** Fixing defects or bugs found in the software.
- **Adaptive Maintenance:** Modifying the software to operate in a changed environment (e.g., new operating system, database).
- **Perfective Maintenance:** Improving the software's performance, maintainability, or other attributes without changing its functionality (e.g., code refactoring, optimization).
- **Preventive Maintenance:** Modifying the software to prevent future problems (e.g., updating documentation, code cleanup).

Documenting these changes is crucial for understanding the software's history, facilitating future maintenance, and ensuring consistency.

Procedure/Steps:

1. **Select an Earlier Developed Module:** Choose a module from Lab 7 (Hospital Management System) that was previously designed or implemented. For instance, the "Patient Registration" module.
2. **Identify a New Feature:** Determine a new feature to add to the selected module. As per the task, a "login system" is specified.
3. **Analyze Impact of New Feature:** Understand how integrating the login system will affect the existing Patient Registration module (e.g., how patient data access will be controlled, user authentication).
4. **Design the New Feature:** Outline the design for the login system, including user authentication, roles (e.g., doctor, nurse, admin), and session management.
5. **Implement the New Feature:** Integrate the login system into the chosen module using the previously used programming language (Java/Python/Cpp).
6. **Test the Modified Module:** Thoroughly test the module to ensure the new feature works correctly and doesn't introduce regressions to existing functionalities.
7. **Document Changes and Maintenance Activities:** Record all modifications made, the reason for the changes (e.g., adding new feature, improving security), the type of maintenance performed (perfective/adaptive), and any challenges encountered.

Practical Work (Example/Diagram/Code):

(Example for a simplified "Patient Registration" module from Lab 7 - conceptual)

Before Modification (Simplified Patient Registration - conceptual)

class Patient:

```
    def __init__(self, patient_id, name, dob):  
        self.patient_id = patient_id  
        self.name = name  
        self.dob = dob
```

```
def display_info(self):  
    return f"Patient ID: {self.patient_id}, Name: {self.name}, DOB: {self.dob}"
```

```
class PatientRegistration:
```

```
    def __init__(self):  
        self.patients = {}
```

```
    def register_patient(self, patient_id, name, dob):  
        if patient_id not in self.patients:  
            patient = Patient(patient_id, name, dob)  
            self.patients[patient_id] = patient  
            return True  
        return False
```

```
    def get_patient(self, patient_id):  
        return self.patients.get(patient_id)
```

```
# Example Usage:
```

```
# pr = PatientRegistration()
```

```
# pr.register_patient("P001", "John Doe", "1990-01-15")
```

```
# print(pr.get_patient("P001").display_info())
```

```
(Example for adding a "Login System" - conceptual and partial implementation)
```

```
# After Modification (Adding a Login System to control access)
```

```
class User:
```

```
    def __init__(self, username, password, role):  
        self.username = username  
        self.password = password # In a real system, passwords would be hashed  
        self.role = role
```

```
class AuthSystem:
```

```
    def __init__(self):  
        self.users = {  
            "admin": User("admin", "adminpass", "admin"),  
            "doctor1": User("doctor1", "docpass", "doctor")  
        }  
        self.current_user = None
```

```
    def login(self, username, password):  
        user = self.users.get(username)  
        if user and user.password == password:  
            self.current_user = user  
            print(f"User {username} logged in successfully as {user.role}.")  
            return True  
        print("Invalid username or password.")  
        return False
```

```
    def logout(self):  
        if self.current_user:  
            print(f"User {self.current_user.username} logged out.")
```

```

        self.current_user = None
    else:
        print("No user is currently logged in.")

    def has_role(self, required_role):
        return self.current_user and self.current_user.role == required_role

class SecurePatientRegistration(PatientRegistration): # Inherit from previous class
    def __init__(self, auth_system):
        super().__init__()
        self.auth_system = auth_system

    def register_patient(self, patient_id, name, dob):
        if self.auth_system.current_user and self.auth_system.has_role("admin"):
            return super().register_patient(patient_id, name, dob)
        print("Permission denied: Only administrators can register patients.")
        return False

    def get_patient(self, patient_id):
        if self.auth_system.current_user and (self.auth_system.has_role("admin") or
self.auth_system.has_role("doctor")):
            return super().get_patient(patient_id)
        print("Permission denied: Only administrators or doctors can view patient info.")
        return None

# Example Usage with new login system:
# auth = AuthSystem()
# secure_pr = SecurePatientRegistration(auth)

# auth.login("doctor1", "docpass")
# secure_pr.register_patient("P002", "Jane Doe", "1985-05-20") # Fails due to role
# auth.logout()

# auth.login("admin", "adminpass")
# secure_pr.register_patient("P002", "Jane Doe", "1985-05-20") # Succeeds
# print(secure_pr.get_patient("P002").display_info())
# auth.logout()

```

Documentation of Changes and Maintenance Activities:

Date	Module Changed	Description of Change	Reason for Change	Type of Maintenance
2025-08-23	Patient Registration	Added a new <code>AuthSystem</code> class to handle user authentication (login/logout) and role-based access control. The <code>SecurePatientRegistration</code> class was created, inheriting from <code>PatientRegistration</code> , to integrate this authentication. Methods like <code>register_patient</code> and <code>get_patient</code> were modified to check user roles before execution.	Enhance security and access control.	Perfective

Date	Module Changed	Description of Change	Reason for Change	Type of Maintenance
2025-08-23	Patient Registration	Implemented basic user roles (admin, doctor) and checked these roles for patient registration and viewing.	Provide granular access permissions.	Perfective
2025-08-23	Patient Registration	Updated <code>register_patient</code> and <code>get_patient</code> to use <code>AuthSystem.current_user</code> and <code>AuthSystem.has_role()</code> for authorization.	Integrate new security mechanism.	Perfective

Conclusion/Observation:

This lab provided a practical understanding of software evolution and maintenance by requiring us to modify an existing module (Patient Registration from Lab 7) to incorporate a new feature (a login system). We observed that even a seemingly small addition can impact existing functionalities and necessitate careful integration. The process highlighted the importance of designing for extensibility and the need for thorough testing after any modification. Furthermore, documenting the changes in a structured maintenance log proved invaluable for tracking modifications, understanding their rationale, and classifying the type of maintenance performed (in this case, primarily perfective maintenance for adding new functionality and improving security). This exercise demonstrated that software development is an ongoing process of evolution and adaptation, requiring continuous maintenance efforts.

Lab-10

Project Management Exercise

Objective: Learn estimation and scheduling techniques in software project management.

Theory:

Software project management involves planning, organizing, and managing resources to develop software efficiently. Key aspects include:

- **Work Breakdown Structure (WBS):** A hierarchical decomposition of the total scope of work to be carried out by the project team to accomplish the project objectives and create the required deliverables. It breaks down project work into smaller, more manageable components.
- **COCOMO Model (Constructive Cost Model):** An algorithmic cost estimation model for software projects. It predicts the effort, cost, and schedule for a software project based on lines of code (or function points) and a set of cost drivers. There are three modes: Organic, Semidetached, and Embedded.
- **Gantt Chart:** A horizontal bar chart widely used in project management to visually represent a project schedule. It illustrates the start and finish dates of the terminal elements and summary elements of a project, and often includes dependencies between tasks.

Procedure/Steps:

1. **Work Breakdown Structure (WBS) for Online Shopping System:**
 - Identify the major phases of an Online Shopping System development (e.g., Requirements, Design, Implementation, Testing, Deployment).
 - Decompose each phase into smaller, manageable tasks and sub-tasks.
 - Present the WBS in a hierarchical structure (e.g., outline format or tree diagram).
2. **Estimate Cost using COCOMO Model:**
 - For the Online Shopping System, make an assumption about its size (e.g., lines of code or estimated function points) and categorize it into one of the COCOMO modes (e.g., Organic, Semidetached, Embedded).
 - Apply the appropriate COCOMO formula (Basic, Intermediate, or Detailed) to estimate the effort (person-months) and schedule (months).
 - Clearly state assumptions made for the calculation.
3. **Prepare Gantt Chart for Project Scheduling:**
 - Based on the WBS, list all identified tasks.
 - Estimate the duration for each task.
 - Identify dependencies between tasks (which tasks must finish before others can start).
 - Create a Gantt chart showing the start and end dates of each task, along with their dependencies.

Practical Work (Example/Diagram/Code):

(Example for Work Breakdown Structure (WBS) for an Online Shopping System)

1. Online Shopping System Development

1.1. Project Initiation & Planning

1.1.1. Define Project Scope & Objectives

1.1.2. Feasibility Study

1.1.3. Resource Planning

1.1.4. Project Schedule (Gantt Chart creation)

1.2. **Requirements Gathering & Analysis**

1.2.1. Functional Requirements (User Stories: Product Search, Add to Cart, Checkout, etc.)

1.2.2. Non-Functional Requirements (Security, Performance, Usability)

1.2.3. Use Case Modeling

1.3. **System Design**

1.3.1. Database Design (Schema for Products, Users, Orders)

1.3.2. Architecture Design (e.g., MVC, Microservices)

1.3.3. UI/UX Design (Wireframes, Mockups)

1.3.4. API Design

1.4. **Implementation**

1.4.1. Frontend Development (User Interface)

1.4.2. Backend Development (Business Logic, API Endpoints)

1.4.3. Database Implementation

1.4.4. Payment Gateway Integration

1.5. **Testing**

1.5.1. Unit Testing

1.5.2. Integration Testing

1.5.3. System Testing (Functional, Performance, Security)

1.5.4. User Acceptance Testing (UAT)

1.6. **Deployment**

1.6.1. Server Setup & Configuration

1.6.2. Application Deployment

1.6.3. Data Migration (if applicable)

1.7. Maintenance & Support

1.7.1. Bug Fixing

1.7.2. Feature Enhancements

1.7.3. System Monitoring

(Example for COCOMO Model Estimation)

Assumptions:

- Project Size: Estimated to be 30,000 Lines of Code (KLOC).
- COCOMO Mode: Organic (Small, simple, well-understood projects; stable development environment).

Basic COCOMO Formula:

Effort (E) = $a * (KLOC)^b$ [Person-Months]

Development Time (D) = $c * (Effort)^d$ [Months]

Coefficients for Organic Mode:

$a = 2.4$, $b = 1.05$, $c = 2.5$, $d = 0.38$

Calculation:

$E = 2.4 * (30)^{1.05}$

$E = 2.4 * 33.35 \approx 80.04$ Person-Months

$D = 2.5 * (80.04)^{0.38}$

$D = 2.5 * 5.42 \approx 13.55$ Months

Result:

- Estimated Effort: Approximately 80 Person-Months
- Estimated Development Time: Approximately 13.5 months

(Example for Gantt Chart - Conceptual representation)

Task ID	Task Name	Duration (Weeks)	Start Date	End Date	Dependencies
1	Project Initiation & Planning	2	Week 1	Week 2	-

Task ID	Task Name	Duration (Weeks)	Start Date	End Date	Dependencies
2	Requirements Gathering & Analysis	4	Week 3	Week 6	1
3	System Design	6	Week 7	Week 12	2
4	Implementation (Frontend)	8	Week 13	Week 20	3
5	Implementation (Backend)	10	Week 13	Week 22	3
6	Database Implementation	4	Week 7	Week 10	3
7	Payment Gateway Integration	3	Week 21	Week 23	5
8	Testing	5	Week 23	Week 27	4, 5, 6, 7
9	Deployment	2	Week 28	Week 29	8
10	Maintenance & Support (Ongoing)	Indefinite	Week 30	-	9

(A visual Gantt chart would show bars for each task, with their lengths representing duration and positions indicating start/end dates. Lines or arrows would indicate dependencies.)

Conclusion/Observation:

This lab provided a comprehensive exercise in software project management by applying key techniques for project estimation and scheduling. We successfully created a hierarchical Work Breakdown Structure (WBS) for an Online Shopping System, which helped in decomposing the complex project into manageable tasks. Using the COCOMO model, we estimated the effort and schedule required for the project based on an assumed size and mode, illustrating how quantitative methods can be used for initial project planning. Finally, developing a Gantt chart allowed us to visualize the project timeline, task durations, and critical dependencies, which is essential for effective project tracking and resource allocation. This lab demonstrated the practical application of these tools in ensuring that software projects are planned, executed, and delivered efficiently within defined constraints.